

Calling a function require you to follow a few conventions related to sending arguments and receiving return values.

Registers `rsi, rdi, ...` used to send integer args.

Return value `rax` register.

`mov r8, 8`

`call printf`

`add r8, rax`

Might have used `r8` for its own internal purpose.

How do we know which register are used by the internal functions?

Registers are partitioned into two categories
→ volatile and non-volatile.

Value stored by the calling function
can be changed by the called
function.

Responsibility of maintaining this data
is on the calling function.

Non-volatile : → Responsibility of saving the
data is on the called function.

non-volatile rnv
volatile .rvo

printf :
push rnv

pop

push rvo
call printf

Structures in assembly: →

What is class and what is structure?

```
struct my_struct {  
    int a;  
    :  
};
```

a = 5 X

```
struct my_struct data;  
data.a = 5;
```

```
struct my_struct  
db a 0
```

a = 5 X

```

struct my_struct
{
    db my_struct.a 0
    dq . . . . . b 5
}

```

Alignment of data in structures: →

```

struct my_struct {
    char a[35];
    int x;
};

```

35
 0-34 35-42
 └──┬──┘
 x

0-34 40-47

Padding is adding additional memory which does not store data, but helps in alignment.

0-34 35-39 40-47
 └──┬──┘ └──┬──┘ └──┬──┘
 char padding int

"align" keyword

```
struct my_struct
```

```
db my_struct.a times 35
```

```
align 5 times → Necessary.
```

```
dq my_struct.x
```

x86-64 processors support backward compatibility
Both 16-bit mode and 32-bit mode.

During boot process, they can only work in
16-bit mode. Why is this so?

Modes of execution → Depends on the value
stored in the control register (CRO)

```
mov cro, 1✓✓
```

When you are booting, execution starts in 16-bit
mode and a specific location of memory.

() Execution jumps and resumes from a
different location in memory.

Computer's OS initially is loaded by a boot
loader.

Boot loader needs to keep the instructions in the appropriate locations in the RAM.

Can we change these modes in our assembly programs?

· Privilege level determines whether these specific instructions can be executed in an assembly program. The privilege level is also stored in the cr0 register.

Dynamic memory allocation in assembly

malloc function in assembly can be used.

$(\text{void} *) \text{malloc}(\text{size_t})$
unsigned int

struct *abc;
abc = malloc((struct abc*) sizeof(my_struct) * 100)

mov rdi, 48

call malloc_{t8}

mov [rax], <data>

1) You need to set up a VM and become familiar with setting up Linux within it.
str: %d", 0x0a, 0

```
mov rsi, rax  
lea rdi, [str]  
printf("%d", rax) call printf
```